Building AI Agent Systems That Keep Your Data Safe

LangChain, LangGraph & Local LLM Deployment

Le Cai
CABS AI Productivity Series

What This Workshop Is About

Understand what AI agents are, how your data flows through them, and how to build one — including locally on your own machine.

This is not a LangChain tutorial.

This is a thinking framework — with working examples.

Workshop Roadmap

1

Concepts

LLM vs Agent
What's the difference?

2

Data Security

Where does your
data go?

3

Hands-On

Build a RAG + Agent
Run locally

What Is an LLM?

A model that predicts the next word, trained on massive text data.

Can do

- Summarize text
- Translate languages
- Answer questions
- Generate code

Cannot do

- Search the internet
- Query a database
- Remember past conversations
- Take actions in the real world

Key property: stateless — every call starts fresh

What Is an Agent?

LLM

A brain that can only talk.

You ask → it answers.

Agent

Brain + hands + eyes.

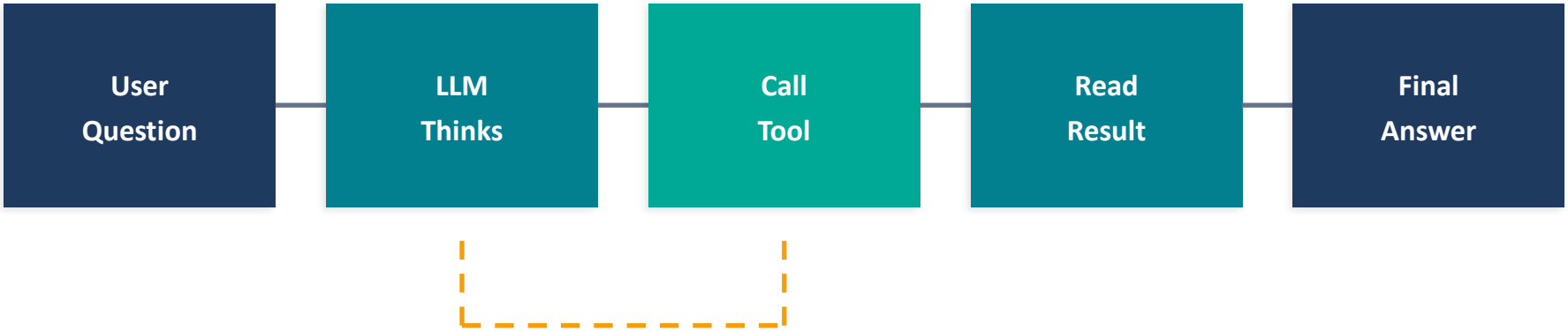
It can decide what to do, then do it.

The LLM is the decision-maker inside the agent.

The agent is the system that can act on those decisions.

The Agent Loop

ReAct = Reasoning + Acting



May loop: think → act → observe → think again

A Biology Example

"What is the latest research on TP53 in pancreatic cancer?"

LLM Alone

Answers from training data.

May be outdated.

Cannot check PubMed.

Cannot verify current status.

Agent

Searches PubMed → reads results

→ looks up TP53 on UniProt

→ checks AlphaFold structure

→ combines all into answer

Same question, very different capability.

What Are Tools?

Just Python functions the agent can call.

Each tool has a description — the LLM reads it to decide when to use it.

UniProt API

Gene / protein info

AlphaFold API

Predicted 3D structures

PubMed Search

Latest publications

BLAST

Sequence alignment

File Reader

Read local data files

Calculator

Math operations

You decide what tools the agent has access to.

Why Are Agents "Unstable"?

The LLM makes decisions probabilistically — it samples from a probability distribution, not a deterministic function.

- Same question → might choose different tools, different order
- This is not a bug — it's a property of the architecture
- Traditional ML: same input → same output (deterministic)
- LLM: same input → probability distribution → sampled output

Analogy: an agent is like a new RA — smart but needs clear SOPs.
Don't expect perfection. Design for graceful failure.

How Do You Control an Agent?

Limit tools

Don't give it tools it doesn't need

Clear descriptions

Tool docstrings are instructions for the LLM

Structured flows

Use LangGraph for deterministic steps

Validate outputs

Check results before acting on them

The Key Question:

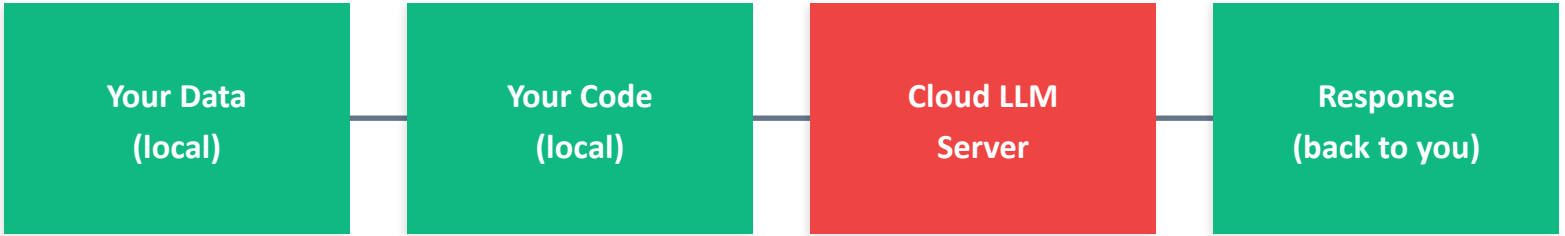
Where Is the LLM?

When you send a prompt to ChatGPT, Claude, or Gemini —

your text goes to their server, gets processed, comes back.

Your prompt, your data, your reasoning: all on someone else's computer.

Data Flow: Cloud LLM



The question is not "is AI safe?" — it's "where does my data go during inference?"

Three Deployment Modes

Public Cloud API

ChatGPT, Gemini free tier

Data leaves your machine

Privacy policy only

Enterprise Cloud

AWS Bedrock, Azure OpenAI

Data leaves, with legal contract

BAA / HIPAA compliance

Local Deployment

Ollama on your machine

Data never leaves

You control everything

When Do You Need Local?

- Patient data / Protected Health Information (PHI)
- Unpublished experimental data
- Proprietary sequences or compounds
- Data under agreements prohibiting third-party transfer
- Air-gapped or restricted network environments

Simple test: "Would I paste this into ChatGPT?"

If no → you need local or enterprise deployment.

The Tradeoff Is Real

	Cloud (Gemini, GPT-4)	Local (Ollama 8B)
Quality	High — frontier models	Lower — smaller models
Tool-calling	Reliable	Less reliable
Speed	Fast (cloud GPU)	Depends on hardware
Privacy	Data leaves machine	Data stays local
Cost	Free tier has limits	Free forever
Internet	Required	Not required (for LLM)

This is a spectrum, not a binary choice. Right answer depends on your data and task.

Live Demo: Ollama

Install → Pull model → Run → Ask a question

No API key. No internet (for the LLM). No data leaving.

[Switch to Terminal]

Hands-On: Three Notebooks, One Story

01

RAG

LLM reads
your documents

Passive reading

02

Agent

LLM calls
external tools

Active action

03

Local

Same agent,
your machine

Data control

Setup: Get Started

1. Open the Colab link (next slide or GitHub repo)

2. Get a free Gemini API key:

   aistudio.google.com → Sign in → Get API Key → Create

3. Run the first two cells: install packages, enter your key

No credit card needed. Takes about 2 minutes.

Notebook 01: RAG — Literature Q&A

Ask questions about 10 organ-on-a-chip papers in natural language.

[Switch to Colab — walk through together]

Notebook 02: Agent — Gene & Structure Lookup

The LLM decides which tool to call based on your question.

Tool 1: UniProt Lookup

Gene name → protein function,
disease associations, subcellular location

Tool 2: AlphaFold Lookup

UniProt ID → predicted 3D structure,
confidence score, visualization link

[Switch to Colab — walk through together]

The One-Line Swap

Cloud:

```
llm = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
```

Local:

```
llm = ChatOllama(model="llama3.1:8b")
```

Everything else — tools, agent, prompts — stays identical.

Data Flow: Cloud vs. Local

Cloud Mode

Your prompt + documents

→ Google's server

→ Response back

Local Mode

Your prompt + documents

→ Ollama on YOUR machine

→ Response (never left)

In both modes, UniProt/AlphaFold API calls go over the internet — but those only send public gene names, not your private data or reasoning.

Live Demo: Local Agent

Running the same gene lookup agent from Notebook 02 —

but with llama3.1:8b on this machine.

Honest about quality tradeoffs:

where it works, where it struggles.

[Switch to Terminal]

What You Learned Today

LLM vs Agent

Understanding the architecture — brain vs. brain + hands

Data Flows

Knowing exactly where your data goes during inference

Three Modes

Cloud → enterprise → local: a spectrum of privacy options

Practical Skills

Built a RAG and an agent you can extend with your own tools

Where to Go Next

- Workshop materials: github.com/lecaibio/cabs-workshop-llm-agents
- Browse models: ollama.com/library
- LangChain docs: python.langchain.com
- Add your own tools: PubMed, BLAST, KEGG, lab inventory
- Open WebUI: ChatGPT-like local interface for Ollama

When you hit errors: paste them into an AI and ask it to explain.

Use AI to learn AI.

Thank You

CABS AI Productivity Series

github.com/lecaibio/cabs-workshop-llm-agents

Q & A